

Sliding Window M Longest Substring Without Repeating Characters

Longest Substring Without Repeating Characters : Given string s , return length of longest substring (contiguous sequence) without repeating characters, Example Input: "abcabcbb" → Output: 3; optimal approach uses sliding window with Hash Map storing last index of characters and moves left pointer when duplicate found to maintain unique window, Time Complexity $O(n)$, Space Complexity $O(\min(n, \text{charset size}))$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Sliding Window Technique** - Used to maintain a window of unique characters that can expand and shrink dynamically
- **Hash Set** - Needed to track which characters are currently in the window for $O(1)$ duplicate detection
- **Hash Map (optional)** - The optimal solution uses a map to store character indices for direct pointer jumps

2. Sliding Window

Intuition

Instead of restarting at every index like brute force, we can keep one **window** that always has *unique* characters.

We expand the window by moving the right pointer.

If we ever see a repeated character, we shrink the window from the left until the duplicate is removed.

This way, the window always represents a valid substring, and we track its maximum size.

It's efficient because each character is added and removed at most once.

Algorithm

1. Create an empty set `charSet` and two pointers:
 - l = left edge of the window
 - r = right edge that moves through the string
2. For each r :
 - While $s[r]$ is already in the set:
 - Remove $s[l]$ from the set and move l right.
 - Add $s[r]$ to the set.
 - Update the result with the window size: $r - l + 1$.
3. Return the maximum window size found.

class Solution:

```
def lengthOfLongestSubstring(self, s: str) -> int:
    charSet = set()
    l = 0
    res = 0

    for r in range(len(s)):
        while s[r] in charSet:
            charSet.remove(s[l])
            l += 1
        charSet.add(s[r])
        res = max(res, r - l + 1)

    return res
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(m)O(m)$

Where nn is the length of the string and mm is the total number of unique characters in the string.

```
func lengthOfLongestSubstring(s string) int {
    charSet := make(map[byte]bool)
    l, res := 0, 0

    for r := 0; r < len(s); r++ {
        for charSet[s[r]] {
            delete(charSet, s[l])
            l++
        }
        charSet[s[r]] = true
        if r - l + 1 > res {
            res = r - l + 1
        }
    }

    return res
}
```

Common Pitfalls

Not Taking the Maximum When Jumping the Left Pointer

In the optimal sliding window approach, when you find a duplicate character, you should move the left pointer to $\max(\text{left}, \text{lastIndex}[\text{char}] + 1)$. Forgetting the max operation can cause the left pointer to move backwards if the duplicate character's last occurrence is before the current window start, leading to incorrect results.

Forgetting to Update the Character's Last Index

After processing each character, you must update its last seen index in the map, regardless of whether it was a duplicate. Failing to update the index means future duplicate checks will reference stale positions, causing incorrect window calculations.

Off-by-One in Window Size Calculation

When calculating the substring length, ensure you use $\text{right} - \text{left} + 1$ since both indices are inclusive. Using $\text{right} - \text{left}$ will undercount the length by one, resulting in an answer that is consistently one less than correct.

Sliding Window E Best Time to Buy And Sell Stock

Best Time to Buy And Sell Stock : Given array prices where $\text{prices}[i]$ is stock price on day i , find maximum profit with at most one transaction (buy once and sell once), Example Input: $[7,1,5,3,6,4] \rightarrow$ Output: 5; optimal approach keeps track of minimum price seen so far and updates maximum profit while scanning once (this is a dynamic sliding minimum problem), Time Complexity $O(n)$, Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Arrays** - Iterating through elements and tracking values
- **Two Pointers** - Using multiple pointers to track buy and sell positions
- **Greedy Algorithms** - Making locally optimal choices (tracking minimum buy price) for global optimization

2. Two Pointers

Intuition

We want to buy at a low price and sell at a higher price that comes **after** it.

Using two pointers helps us track this efficiently:

- l is the **buy day** (looking for the lowest price)
- r is the **sell day** (looking for a higher price)

If the price at r is higher than at l , we can make a profit — so we update the maximum.

If the price at r is lower, then r becomes the new l because a cheaper buying price is always better.

By moving the pointers this way, we scan the list once and always keep the best buying opportunity.

Algorithm

1. Set two pointers:
 - $l = 0$ (buy day)
 - $r = 1$ (sell day)
 - $\text{maxP} = 0$ to track maximum profit
2. While r is within the array:
 - If $\text{prices}[r] > \text{prices}[l]$, compute the profit and update maxP .
 - Otherwise, move l to r (we found a cheaper buy price).
 - Move r to the next day.
3. Return maxP at the end.

class Solution:

```
def maxProfit(self, prices: List[int]) -> int:
    l, r = 0, 1
    maxP = 0

    while r < len(prices):
        if prices[l] < prices[r]:
            profit = prices[r] - prices[l]
            maxP = max(maxP, profit)
        else:
            l = r
        r += 1
    return maxP
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(1)O(1)$

func maxProfit(prices []int) int {

```
    l, r := 0, 1
    maxP := 0

    for r < len(prices) {
        if prices[l] < prices[r] {
```

```

        profit := prices[r] - prices[l]
        if profit > maxP {
            maxP = profit
        }
    } else {
        l = r
    }
    r++
}
return maxP
}

```

Common Pitfalls

Selling Before Buying

The sell day must come after the buy day. Calculating $\text{prices}[i] - \text{prices}[j]$ where $j > i$ means you're selling in the past, which is invalid.

Wrong: selling before buying

```

for i in range(len(prices)):
    for j in range(i): # j < i means selling earlier
        profit = prices[j] - prices[i] # Backwards

```

Returning Negative Profit

If prices only decrease, the maximum profit is 0 (don't trade), not a negative number. Always ensure the result is at least 0.

Wrong: can return negative

```

return maxPrice - minPrice # Could be negative if maxPrice found before minPrice

```

Sliding Window M Longest Repeating Character Replacement

Longest Repeating Character Replacement : Given string s and integer k , find length of longest substring that can be formed by replacing at most k characters to make all characters same, Example Input: $s = \text{"AABABBA"} , k = 1 \rightarrow$ Output: 4; optimal approach uses sliding window maintaining frequency count of characters and tracking max frequency in window, window valid if $(\text{window size} - \text{maxFreq} \leq k)$, Time Complexity $O(n)$, Space Complexity $O(1)$ fixed alphabet.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Sliding Window** - The optimal solution uses a variable-size window that expands and contracts based on validity conditions
- **Hash Map / Frequency Counting** - Tracking character frequencies within the current window to determine the most frequent character
- **Two Pointers** - Managing left and right pointers to define and adjust the window boundaries

3. Sliding Window (Optimal)

Intuition

We want the longest window where we can make all characters the same using at most k replacements.

The key insight is that the window is valid as long as:

window size – count of the most frequent character $\leq k$

Why?

Because the characters that *aren't* the most frequent are the ones we would need to replace.

So while expanding the window, we track:

- the frequency of each character,
- the most frequent character inside the window (maxf).

If the window becomes invalid, we shrink it from the left.

This gives us one clean sliding window pass.

Algorithm

1. Create a frequency map count and initialize $l = 0$, $\text{maxf} = 0$, and $\text{res} = 0$.
2. Move the right pointer r across the string:
 - Update the frequency of $s[r]$.
 - Update maxf with the highest frequency seen so far.
3. If the window is invalid ($\text{window size} - \text{maxf} > k$):
 - Shrink the window from the left and adjust counts.
4. Update the result with the valid window size.
5. Return res at the end.

class Solution:

```

def characterReplacement(self, s: str, k: int) -> int:
    count = {}
    res = 0

    l = 0

```

```

maxf = 0
for r in range(len(s)):
    count[s[r]] = 1 + count.get(s[r], 0)
    maxf = max(maxf, count[s[r]])

    while (r - l + 1) - maxf > k:
        count[s[l]] -= 1
        l += 1
    res = max(res, r - l + 1)

return res

```

Expand

Time & Space Complexity

- Time complexity: $O(n)$
- Space complexity: $O(m)$

Where n is the length of the string and m is the total number of unique characters in the string.

```

func characterReplacement(s string, k int) int {
    count := make(map[byte]int)
    res, l, maxf := 0, 0, 0

    for r := 0; r < len(s); r++ {
        count[s[r]]++
        if count[s[r]] > maxf {
            maxf = count[s[r]]
        }

        for (r - l + 1) - maxf > k {
            count[s[l]]--
            l++
        }

        if r - l + 1 > res {
            res = r - l + 1
        }
    }

    return res
}

```

Common Pitfalls

Not Updating maxf Correctly

The variable `maxf` tracks the maximum frequency of any character in the current window. A common mistake is recalculating the maximum by iterating through all counts after shrinking the window. In the optimal solution, you do not need to decrease `maxf` when shrinking because keeping a stale (higher) `maxf` only makes the window condition stricter, which is still correct. However, misunderstanding this can lead to unnecessary complexity or bugs.

Shrinking the Window Too Aggressively

When the window becomes invalid ($\text{window size} - \text{maxf} > k$), you only need to shrink it enough to make it valid again. A common error is resetting the left pointer too far or not updating character counts properly when moving the left pointer. Make sure to decrement the count of `s[l]` before incrementing `l`.

Confusing Window Size Calculation

The window size is $r - l + 1$, not $r - l$. This off-by-one error is frequent and leads to incorrect validity checks. For example, if $l = 0$ and $r = 2$, the window contains 3 characters, not 2. Always double-check your window size formula in the condition $(r - l + 1) - \text{maxf} \leq k$.

Sliding Window M Maximum Number of Vowels in a Substring of Given Length

Maximum Number of Vowels in a Substring of Given Length : Given string `s` and integer `k`, find maximum number of vowels (characters a,e,i,o,u) in any substring of length `k`, Example Input: `s="abciiidef"`, `k=3` → Output: 3; optimal approach uses fixed-size sliding window adding right character and removing left character while maintaining vowel count, Time Complexity $O(n)$, Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Sliding Window Technique** - The optimal solution maintains a fixed-size window that slides across the string
- **Hash Set / Set Operations** - Used for $O(1)$ vowel membership checking
- **Prefix Sum** - An alternative approach uses cumulative sums to answer range queries efficiently
- **Bit Manipulation (optional)** - Advanced optimization uses bitmasks for faster vowel checking

3. Sliding Window

Intuition

Instead of storing prefix sums, we can maintain a running count of vowels in the current window. As we slide the window right:

- Add 1 if the new character entering the window is a vowel.
- Subtract 1 if the character leaving the window is a vowel.

This gives us $O(n)$ time with $O(1)$ extra space.

Algorithm

1. Initialize a vowel count `cnt` and result `res` to 0.
2. Use two pointers: `l` (left) starts at 0, `r` (right) iterates through the string.
3. For each character at `r`:
 - If it is a vowel, increment `cnt`.
 - If the window size exceeds `k`, check if `s[l]` is a vowel and decrement `cnt` if so, then increment `l`.
 - Update `res` with the maximum of `res` and `cnt`.
4. Return `res`.

class Solution:

```
def maxVowels(self, s: str, k: int) -> int:
    vowel = {'a', 'e', 'i', 'o', 'u'}

    l = cnt = res = 0
    for r in range(len(s)):
        cnt += 1 if s[r] in vowel else 0
        if r - l + 1 > k:
            cnt -= 1 if s[l] in vowel else 0
            l += 1
        res = max(res, cnt)
    return res
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(1)O(1)$ extra space.

Common Pitfalls

Forgetting to Handle the Initial Window

A common mistake is updating the result before the window reaches size `k`. This can cause the algorithm to report incorrect maximum values from incomplete windows. Always ensure the window has exactly `k` elements before comparing with the result.

Not Removing the Outgoing Element Correctly

When sliding the window, forgetting to subtract the vowel count of the element leaving the window (at position `l`) leads to an ever-increasing count. The window must maintain exactly `k` elements, so when adding a new element on the right, the leftmost element must be removed from the count if the window exceeds size `k`.

Case Sensitivity with Vowels

If the input can contain uppercase letters, failing to handle case sensitivity causes vowels like 'A' or 'E' to be missed. Either convert the string to lowercase before processing or include both cases in the vowel set.

Sliding Window M Permutation In String

Permutation In String : Given strings `s1` and `s2`, return true if `s2` contains a permutation (rearranged order) of `s1` as substring, Example Input: `s1="ab"`, `s2="eidbaooo"` → Output: true; optimal approach uses sliding window of size `len(s1)` with frequency arrays and compares counts efficiently, Time Complexity $O(n)$, Space Complexity $O(1)$ fixed alphabet.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Hash Tables / Frequency Counting** - Tracking character counts to compare substrings
- **Sliding Window Technique** - Efficiently checking fixed-size substrings without rebuilding counts
- **String Manipulation** - Working with substrings and character comparisons
- **Sorting** - Used in the brute-force approach to compare permutations

3. Sliding Window

Intuition

Since a permutation of `s1` must have the **same character counts**, we can use a fixed-size sliding window over `s2` whose length is exactly `len(s1)`.

We maintain two frequency arrays:

- one for `s1`
- one for the current window in `s2`

If these two arrays ever match, the window is a valid permutation.

As we slide the window forward, we update counts by removing the left character and adding the new right character — no need to rebuild the counts

each time.

This makes the solution fast and efficient.

Algorithm

1. If s1 is longer than s2, return false.
2. Build character frequency arrays for:
 - s1
 - the first window of s2 of size len(s1)
3. Count how many positions match between the two arrays (matches).
4. Slide the window from left to right across s2:
 - At each step, add the new right character and update counts/matches.
 - Remove the left character and update counts/matches.
 - If at any time matches == 26, return true.
5. After finishing the loop, return whether matches == 26.

class Solution:

```
def checkInclusion(self, s1: str, s2: str) -> bool:
    if len(s1) > len(s2):
        return False

    s1Count, s2Count = [0] * 26, [0] * 26
    for i in range(len(s1)):
        s1Count[ord(s1[i]) - ord('a')] += 1
        s2Count[ord(s2[i]) - ord('a')] += 1

    matches = 0
    for i in range(26):
        matches += (1 if s1Count[i] == s2Count[i] else 0)

    l = 0
    for r in range(len(s1), len(s2)):
        if matches == 26:
            return True

        index = ord(s2[r]) - ord('a')
        s2Count[index] += 1
        if s1Count[index] == s2Count[index]:
            matches += 1
        elif s1Count[index] + 1 == s2Count[index]:
            matches -= 1

        index = ord(s2[l]) - ord('a')
        s2Count[index] -= 1
        if s1Count[index] == s2Count[index]:
            matches += 1
        elif s1Count[index] - 1 == s2Count[index]:
            matches -= 1

        l += 1
    return matches == 26
```

Expand

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(1)O(1)$

```
func checkInclusion(s1 string, s2 string) bool {
    if len(s1) > len(s2) {
        return false
    }

    s1Count := make([]int, 26)
    s2Count := make([]int, 26)
    for i := 0; i < len(s1); i++ {
        s1Count[s1[i]-'a']++
        s2Count[s2[i]-'a']++
    }

    matches := 0
    for i := 0; i < 26; i++ {
```

```

        if s1Count[i] == s2Count[i] {
            matches++
        }
    }

    l := 0
    for r := len(s1); r < len(s2); r++ {
        if matches == 26 {
            return true
        }

        index := s2[r] - 'a'
        s2Count[index]++
        if s1Count[index] == s2Count[index] {
            matches++
        } else if s1Count[index]+1 == s2Count[index] {
            matches--
        }

        index = s2[l] - 'a'
        s2Count[index]--
        if s1Count[index] == s2Count[index] {
            matches++
        } else if s1Count[index]-1 == s2Count[index] {
            matches--
        }
        l++
    }
    return matches == 26
}

```

Common Pitfalls

Not Checking Length Before Proceeding

If s1 is longer than s2, no permutation of s1 can exist as a substring of s2. Forgetting this early check leads to unnecessary computation or index errors when initializing the sliding window.

Incorrect Window Size

The sliding window must be exactly the length of s1. A common mistake is using a variable-size window or forgetting to remove the leftmost character when adding a new right character. This breaks the frequency comparison and produces incorrect results.

Mishandling the Matches Counter

When updating the frequency counts during window sliding, the matches counter must be carefully updated. A character transition from matching to non-matching should decrement matches, and vice versa. Off-by-one errors in these conditions (e.g., checking == vs +1 or -1) cause the algorithm to miss valid windows or report false positives.

Sliding Window M Minimum Size Subarray Sum

Minimum Size Subarray Sum : Given positive integer array nums and integer target, return minimal length of contiguous subarray whose sum $\geq$ target (contiguous means elements next to each other), Example Input: target=7, nums=[2,3,1,2,4,3] → Output: 2 because [4,3]; optimal approach uses sliding window expanding right to increase sum and shrinking left while sum $\geq$ target to minimize length, Time Complexity $O(n)$, Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Sliding Window Technique** - Dynamically adjusting a window of elements by moving left and right pointers to find optimal subarrays
- **Prefix Sum Arrays** - Precomputing cumulative sums to enable $O(1)$ range sum queries
- **Binary Search** - Efficiently finding target values or boundaries in sorted data in $O(\log n)$ time

2. Sliding Window

Intuition

Since all elements are positive, we can use a sliding window approach. We expand the window by moving the right pointer to increase the sum. Once the sum meets or exceeds the target, we try to shrink the window from the left to find the minimum length. This works because removing elements from the left will only decrease the sum, and we want the smallest window that still satisfies the condition.

Algorithm

1. Initialize l = 0, total = 0, and res = infinity.
2. Iterate r from 0 to n-1:
 - Add nums[r] to total.
 - While total $\geq$ target:

- Update res with the minimum of res and $r - l + 1$.
- Subtract `nums[l]` from total and increment `l`.

3. Return 0 if res is infinity, otherwise return res.

class Solution:

```
def minSubArrayLen(self, target: int, nums: List[int]) -> int:
    l, total = 0, 0
    res = float("inf")

    for r in range(len(nums)):
        total += nums[r]
        while total >= target:
            res = min(r - l + 1, res)
            total -= nums[l]
            l += 1

    return 0 if res == float("inf") else res
```

Time & Space Complexity

- Time complexity: $O(n)$
- Space complexity: $O(1)$ extra space.

func minSubArrayLen(target int, nums []int) int {

```
    l, total := 0, 0
    res := len(nums) + 1

    for r := 0; r < len(nums); r++ {
        total += nums[r]
        for total >= target {
            if r-l+1 < res {
                res = r - l + 1
            }
            total -= nums[l]
            l++
        }
    }

    if res == len(nums)+1 {
        return 0
    }
    return res
}
```

Common Pitfalls

Using Strict Equality Instead of Greater-Than-or-Equal

The problem asks for sum greater than or equal to target, not exactly equal. A common bug is checking if `(sum == target)` instead of if `(sum >= target)`. This causes the algorithm to miss valid subarrays where the sum exceeds the target, often returning 0 when a valid answer exists.

Not Handling the No-Solution Case

When no subarray sums to at least target, you must return 0. Initializing `res` to $n + 1$ or infinity and forgetting to check this at the end leads to returning invalid values. Always verify whether `res` was ever updated before returning it.

Shrinking the Window Too Aggressively

In the sliding window approach, some implementations shrink the window by moving the left pointer multiple times in a single iteration without rechecking the sum condition. The correct approach uses a while loop that continues shrinking only as long as `sum >= target`, updating the minimum length at each valid position.

Sliding Window E Contains Duplicate II

Contains Duplicate II : Given integer array `nums` and integer `k`, determine if there exist two distinct indices `i` and `j` such that `nums[i] = nums[j]` and $|i - j| \leq k$ (distance constraint means indices difference at most `k`), Example Input: `nums = [1,2,3,1]`, `k = 3` → Output: true; optimal approach uses a Hash Set representing current sliding window of size at most `k`, removing element when window exceeds `k`, checking membership in $O(1)$, Time Complexity $O(n)$, Space Complexity $O(k)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Hash Map / Hash Set** - Used for $O(1)$ lookups to track element indices or check for duplicates within a window
- **Sliding Window Technique** - Maintaining a fixed-size window of elements as you iterate through the array

3. Hash Set

Intuition

We only need to check for duplicates within a sliding window of size k . Using a hash set, we maintain exactly the elements in the current window. If a new element already exists in the set, we found a duplicate within distance k . We slide the window by removing the leftmost element when the window exceeds size k .

Algorithm

1. Create an empty hash set to represent the sliding window.
2. Use two pointers L and R , with R iterating through the array.
3. If the window size $R - L$ exceeds k , remove $\text{nums}[L]$ from the set and increment L .
4. If $\text{nums}[R]$ is already in the set, return true.
5. Add $\text{nums}[R]$ to the set.
6. If no duplicate is found, return false.

class Solution:

```
def containsNearbyDuplicate(self, nums: List[int], k: int) -> bool:
    window = set()
    L = 0

    for R in range(len(nums)):
        if R - L > k:
            window.remove(nums[L])
            L += 1
        if nums[R] in window:
            return True
        window.add(nums[R])

    return False
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(\min(n,k))O(\min(n,k))$

Where n is the size of the array nums and k is the maximum distance between two equal numbers.

```
func containsNearbyDuplicate(nums []int, k int) bool {
    window := make(map[int]bool)
    L := 0

    for R := 0; R < len(nums); R++ {
        if R-L > k {
            delete(window, nums[L])
            L++
        }
        if window[nums[R]] {
            return true
        }
        window[nums[R]] = true
    }

    return false
}
```

Common Pitfalls

Off-by-One Error in Distance Check

The condition requires the distance to be "at most k ", meaning $\text{abs}(i - j) \leq k$, not $< k$. Using strict inequality will miss valid pairs that are exactly k positions apart.

Wrong: Using strict inequality

if $\text{nums}[i]$ in mp and $i - \text{mp}[\text{nums}[i]] < k$: # Misses distance == k

Correct: Use $\leq$ for "at most k "

if $\text{nums}[i]$ in mp and $i - \text{mp}[\text{nums}[i]] \leq k$:

Storing All Indices Instead of Most Recent

The hash map only needs to store the most recent index for each value. Storing all indices wastes memory and complicates the logic. Since we iterate left to right, we only care about the closest previous occurrence.

Sliding Window M Max Consecutive Ones III

Max Consecutive Ones III : Given binary array nums and integer k , return maximum number of consecutive 1s if you can flip at most k zeros (flip means change 0 to 1), Example Input: $\text{nums}=[1,1,1,0,0,0,1,1,1,1,0]$, $k=2 \rightarrow$ Output: 6; optimal approach uses sliding window tracking number of zeros in window and shrinking left when zero count exceeds k , Time Complexity $O(n)$, Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Arrays** - Understanding how to traverse and manipulate array elements
- **Sliding Window Technique** - Maintaining a dynamic window over a contiguous subarray while tracking specific conditions
- **Two Pointers** - Using left and right pointers to efficiently process subarrays without nested iteration
- **Prefix Sum** - Precomputing cumulative sums to enable $O(1)$ range queries
- **Binary Search** - Finding target values or boundaries in sorted data in $O(\log n)$ time

3. Sliding Window

Intuition

A sliding window provides the optimal approach. We maintain a window that can contain at most k zeros. As we expand the right boundary, we decrement k for each zero encountered. When k goes negative, the window has too many zeros, so we shrink from the left until the window is valid again. The maximum window size seen during this process is our answer. Each element is processed at most twice, yielding linear time complexity.

Algorithm

1. Initialize l (left pointer) and res (result) to 0.
2. Iterate through the array with r (right pointer):
 - If $nums[r]$ is 0, decrement k .
 - While $k < 0$ (window is invalid):
 - If $nums[l]$ is 0, increment k (restore the flip allowance).
 - Move l forward.
 - Update res with $r - l + 1$.
3. Return res .

class Solution:

```
def longestOnes(self, nums: List[int], k: int) -> int:
    l = res = 0
    for r in range(len(nums)):
        k -= (1 if nums[r] == 0 else 0)
        while k < 0:
            k += (1 if nums[l] == 0 else 0)
            l += 1
        res = max(res, r - l + 1)
    return res
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(1)O(1)$

func longestOnes(nums []int, k int) int {

```
    l, res := 0, 0
    for r := 0; r < len(nums); r++ {
        if nums[r] == 0 {
            k--
        }
        for k < 0 {
            if nums[l] == 0 {
                k++
            }
            l++
        }
        if r-l+1 > res {
            res = r - l + 1
        }
    }
    return res
}
```

Common Pitfalls

Modifying k Without Restoring It

When using k directly as a counter (decrementing when encountering zeros), remember that this mutates the input parameter. If you need to use k again later or if the problem requires preserving it, use a separate variable. Additionally, when shrinking the window, you must restore k by incrementing it when a zero leaves the window.

Incorrect Window Shrinking Condition

The window should shrink when $k < 0$, meaning we have flipped more zeros than allowed. A common mistake is using $k == 0$ as the shrinking condition, which prevents the window from ever containing k zeros. The window is valid as long as $k \geq 0$; only shrink when it becomes negative.

Forgetting to Handle Edge Cases

When k equals or exceeds the number of zeros in the array, the answer is simply the length of the entire array. While the sliding window approach handles this naturally, the brute force approach might have issues if not carefully implemented. Also, handle the case when the array is empty or contains only ones.

Sliding Window E Maximum Average Subarray I

Maximum Average Subarray I: Given array and integer k , find contiguous subarray of length k with maximum average, approach sliding window maintaining window sum of size k , optimal time $O(n)$, space $O(1)$.

Example:

Input: `nums=[1,12,-5,-6,50,3], k=4`

Output: `12.75`

Sliding Window E Number of Recent Calls

Number of Recent Calls: Design a class that counts number of requests received in the last 3000 milliseconds (millisecond = one thousandth of a second) where each call provides a timestamp t and you must return how many calls happened in interval $[t-3000, t]$, approach using a queue (FIFO data structure: First-In-First-Out) storing timestamps and removing from front while $\text{timestamp} < t-3000$, optimal time complexity amortized $O(1)$ per call because each timestamp is added and removed once, space $O(n)$ in worst case where n = number of calls within 3000ms window.

Example:

Input: `["RecentCounter","ping","ping","ping","ping"]`

`[[],[1],[100],[3001],[3002]]`

Output: `[null,1,2,3,3]`
